Лекция 8. BI, визуальная аналитика и мониторинг

Технологии и инструментарий анализа больших данных

Курс ТИАБД



Раздел

Введение



Тема восьмой лекции

В предыдущих лекциях мы прошли путь от источников данных до хранения, Spark, машинного обучения, NLP и CV. Теперь необходимо замкнуть систему двумя вопросами.

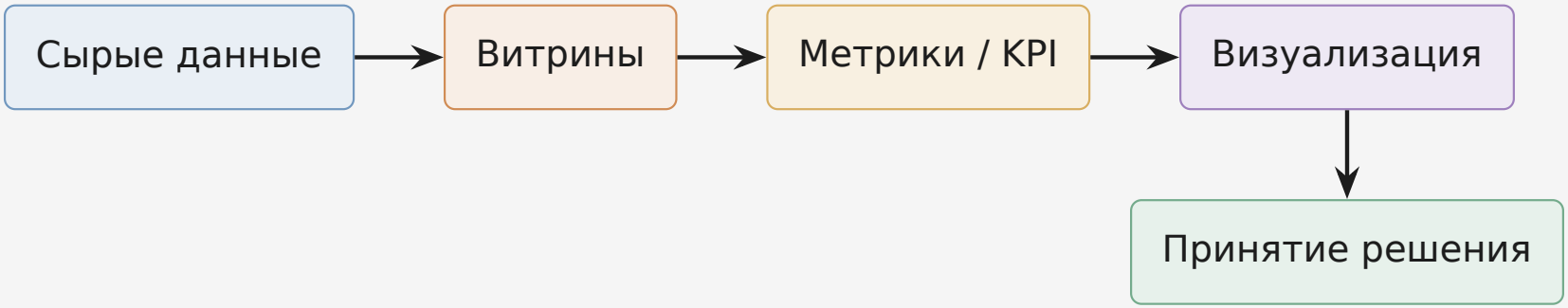
Два практических вопроса финального слоя

- Как превратить результаты обработки данных в информацию для принятия решений?
- Как понять, что сама информационная система работает корректно?

Первый вопрос приводит к **Business Intelligence** и визуальной аналитике. Второй приводит к **мониторингу**, Prometheus и Grafana.



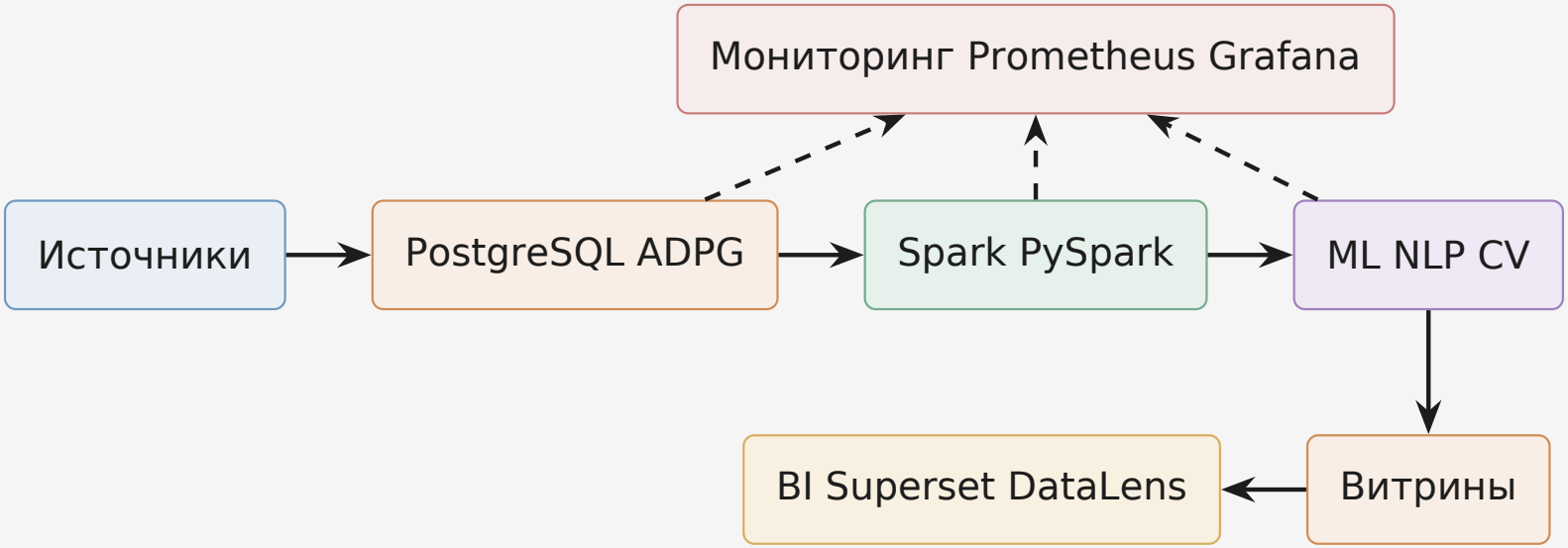
Логика финального слоя



Смысл схемы

Визуализация возникает не вместо подготовки данных, а после того, как определена аналитическая задача, сформирована витрина и согласовано значение показателя.

Место лекции в архитектуре курса



BI смотрит на предметную область. Мониторинг смотрит на вычислительную систему, которая эту предметную область обрабатывает.

Две разные аналитики на одном экране

BI – вопрос о бизнесе

- как меняется выручка;
- какие категории растут;
- где падает конверсия;
- каков средний чек;
- какие регионы требуют внимания.

Мониторинг – вопрос о системе

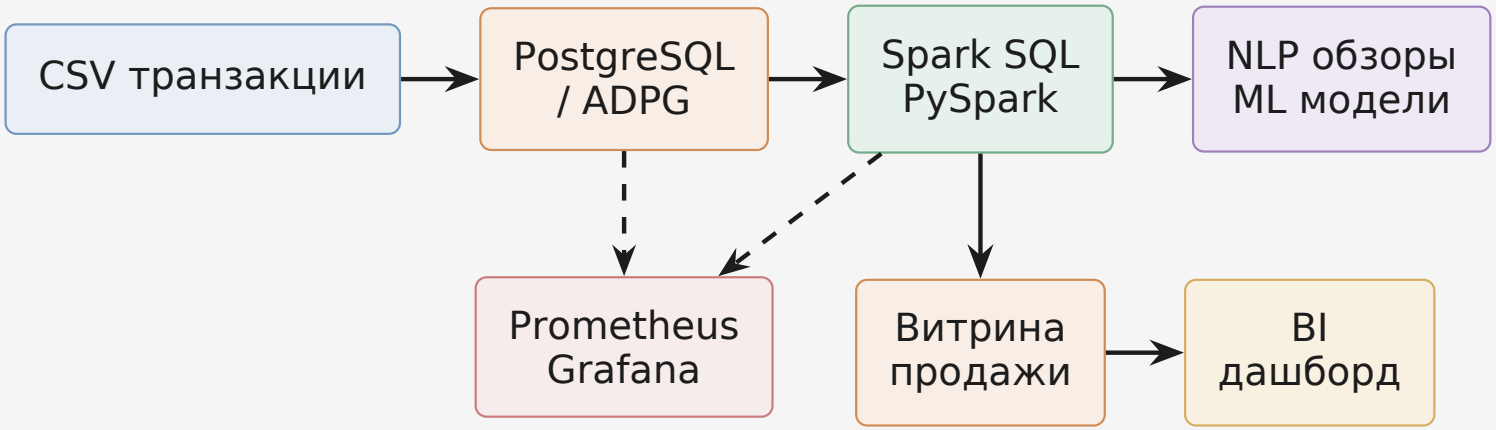
- работает ли PostgreSQL;
- хватает ли CPU и RAM;
- не растёт ли latency;
- не падают ли Spark-задачи;
- сработало ли предупреждение.

На поверхности оба подхода используют графики и дашборды. Но объект наблюдения, смысл метрик и реакция пользователя принципиально различаются.



Сценарий – электронная коммерция

Предположим, что у нас есть учебная аналитическая система.



Один и тот же проект порождает два класса дашбордов: бизнес-дашборды и технические дашборды.



Раздел

Business Intelligence



Что такое Business Intelligence

Business Intelligence — это совокупность процессов, моделей данных и инструментов, которые преобразуют накопленные данные в информацию для анализа и принятия решений.

Ключевая особенность BI

BI почти всегда работает не с отдельной строкой исходной таблицы, а с агрегированной и семантически определённой картиной.

Если строки исходной таблицы обозначить как

$$\{x_1, x_2, \ldots, x_n\},$$

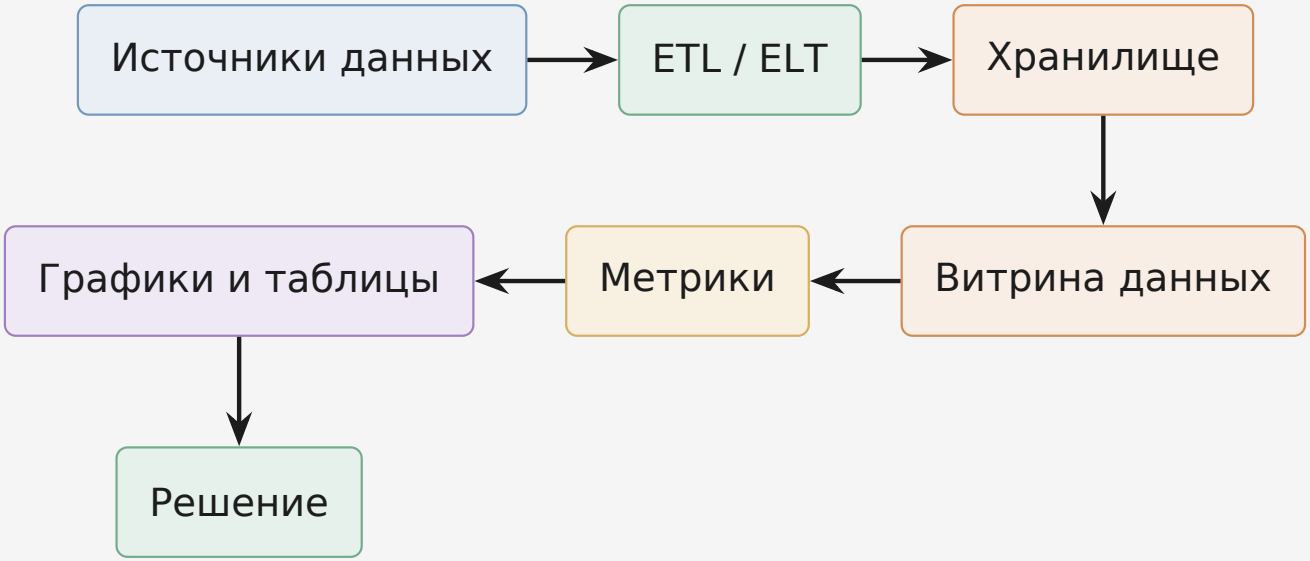
то BI обычно интересует аналитическая функция

$$A(x_1, x_2, \ldots, x_n),$$

например сумма, среднее, доля, темп роста, ранжирование или отклонение от плана.



BI как цепочка преобразования смысла



Важно

График является последним звеном длинной цепочки. Если модель данных противоречива, дашборд лишь делает противоречие визуально заметным.

Витрина данных как основание BI

Data Mart — тематически ограниченная модель данных, подготовленная под конкретный класс аналитических задач.

Исходные таблицы

- `orders`;
- `order_items`;
- `products`;
- `customers`;
- `sellers`;
- `reviews`.

Витрина продаж

- дата заказа;
- регион клиента;
- категория товара;
- продавец;
- цена;
- оценка отзыва.

OLTP схема $\neq$ BI семантическая модель



Почему BI не должен жить только в интерфейсе

BI-инструмент позволяет строить графики, фильтры и дашборды, но он не должен становиться скрытым хранилищем всей предметной логики.

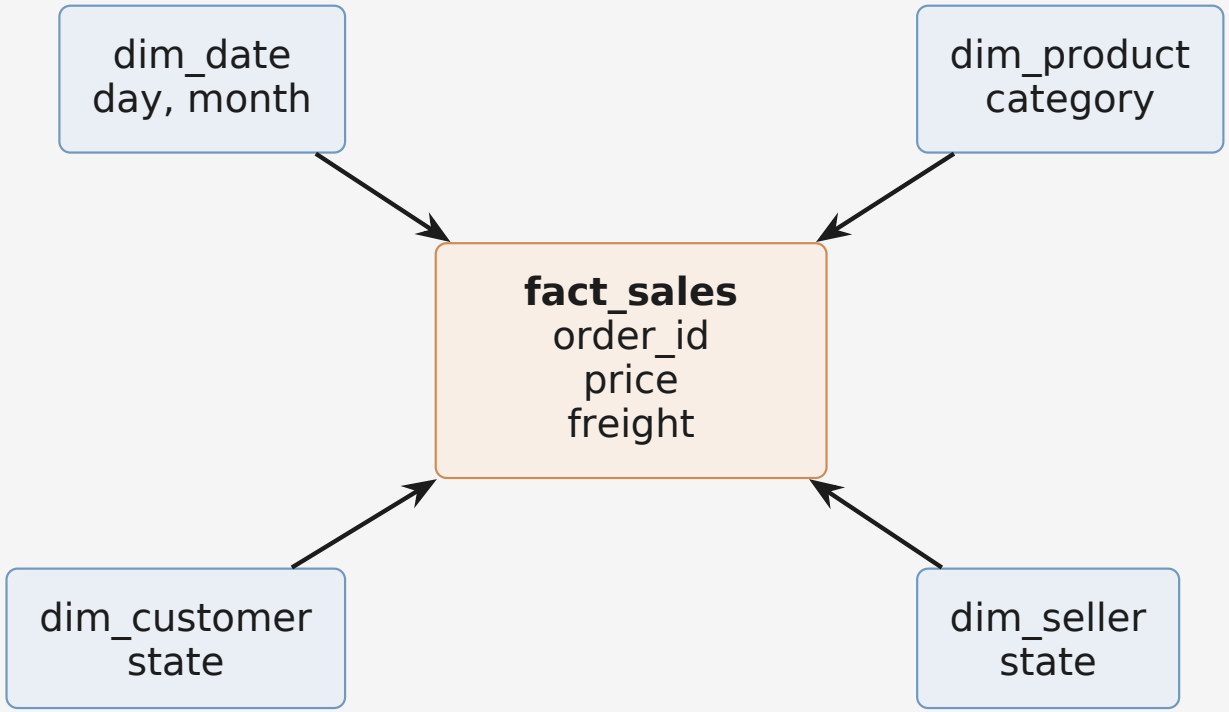
Хорошая практика

- существенные JOIN и очистку данных выполнять в воспроизводимом SQL или Spark-процессе;
- метрики фиксировать в едином слое определений;
- дашборд использовать для анализа, а не для исправления данных;
- различать вычисление показателя и его визуальное представление.

Чем больше существенной логики спрятано в настройках графика, тем сложнее проверять корректность аналитики.



Звёздная схема для аналитики продаж



Смысл звёздной схемы

Фактовая таблица хранит измеримые события, а таблицы измерений описывают оси анализа – время, товар, клиент, продавец, регион.

Метрика, мера и KPI

Метрика

Количественно определённый показатель некоторого свойства процесса или объекта.

Мера

Числовое поле или вычисление, которое агрегируется в BI: сумма, среднее, количество, минимум, максимум.

KPI

Показатель, напрямую связанный с целевым критерием эффективности. Не всякая метрика является KPI.

Например, количество строк в таблице является технической метрикой набора данных, но обычно не является ключевым показателем эффективности бизнеса.



Формальные определения показателей

Метрика должна иметь однозначное определение.
Например, для системы продаж можно определить:

$$GMV = \sum_{i=1}^{N} Price_i,$$

$$Orders = |\{order_id\}|,$$

$$AOV = \frac{Revenue}{Orders}.$$

Главная инженерная проблема

Если две команды считают GMV по-разному, проблема находится не в визуализации, а в семантическом слое данных.



Измерения и меры

Измерения

Качественные оси, по которым группируются и фильтруются данные.

- дата;
- регион;
- категория;
- продавец;
- клиентский сегмент.

Меры

Числовые величины, которые агрегируются.

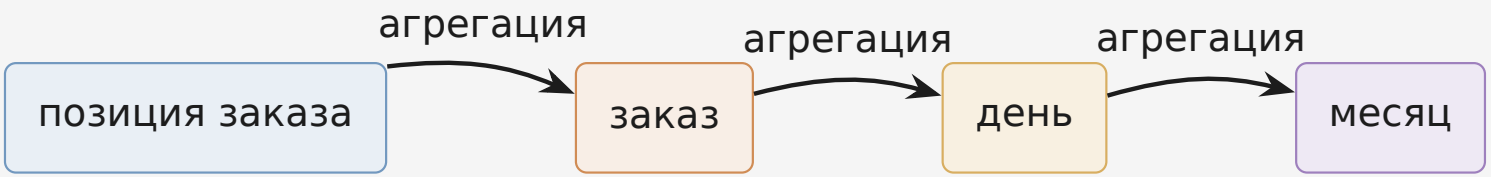
- выручка;
- количество заказов;
- средний чек;
- доставка;
- средняя оценка.

BI-график почти всегда задаётся парой: **измерение + мера**.



Гранулярность данных

Гранулярность показывает, что именно соответствует одной строке витрины.



Почему это важно

Одна и та же мера даёт разные значения в зависимости от уровня детализации. Средний чек нельзя механически усреднять по уже усреднённым значениям без учёта весов.



Пример SQL-витрины продаж

```sql
CREATE MATERIALIZED VIEW mart_sales AS
SELECT
   o.order_id,
   date_trunc('day', o.order_purchase_timestamp) AS order_day,
   c.customer_state,
   s.seller_state,
   p.product_category_name AS category,
   oi.price,
   oi.freight_value,
   r.review_score
FROM orders o
JOIN order_items oi ON oi.order_id = o.order_id
JOIN products p ON p.product_id = oi.product_id
JOIN customers c ON c.customer_id = o.customer_id
JOIN sellers s ON s.seller_id = oi.seller_id
LEFT JOIN order_reviews r ON r.order_id = o.order_id;
```

В промышленном контуре эта логика может быть реализована в DWH, ELT-процессе или Spark-задаче.



Пример агрегата для BI

```sql
SELECT
   date_trunc('month', order_day) AS month,
   category,
   SUM(price) AS gmv,
   COUNT(DISTINCT order_id) AS orders,
   SUM(price) / COUNT(DISTINCT order_id) AS aov
FROM mart_sales
GROUP BY 1, 2
ORDER BY month, gmv DESC;
```

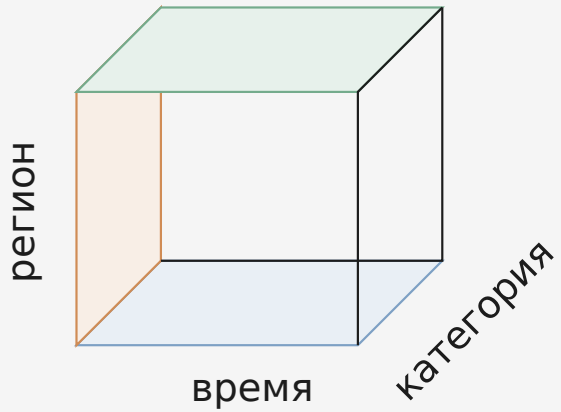
Что получает BI-инструмент

Не произвольный набор строк, а уже осмысленную таблицу: месяц, категория, GMV, число заказов, средний чек.



OLAP – анализ многомерных данных

OLAP — класс аналитической обработки, ориентированный на чтение, агрегацию, группировку и исследование исторических данных.



$$\text{мера} = f(\text{время}, \text{регион}, \text{категория}, \ldots)$$



Основные операции OLAP

Навигация по данным

- **slice** — срез по одному значению измерения;
- **dice** — подкуб по нескольким условиям;
- **pivot** — перестановка осей анализа.

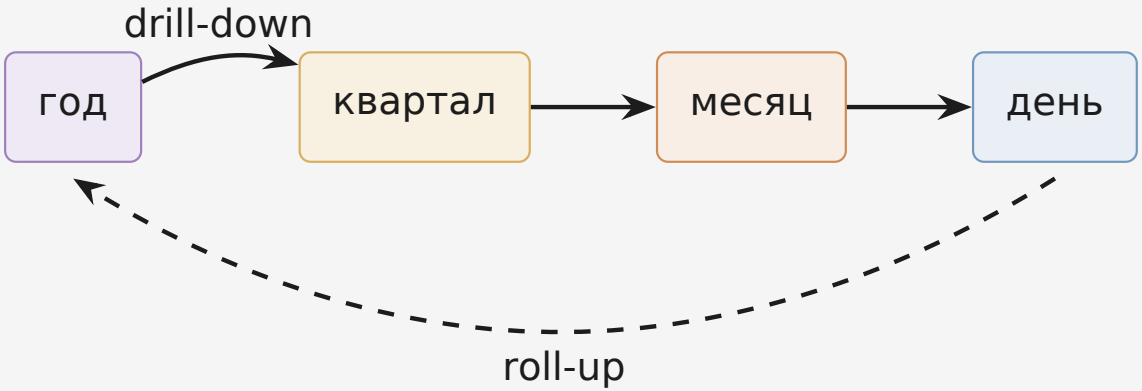
Изменение детализации

- **drill-down** — переход к более детальному уровню;
- **roll-up** — переход к более общему уровню;
- **aggregation** — вычисление меры на новом уровне.

OLAP полезен тогда, когда аналитик не просто смотрит одну диаграмму, а исследует данные по нескольким измерениям.



Drill-down и roll-up



Пример

Сначала видим падение GMV за квартал. Затем проваливаемся до месяцев, потом до недель, затем до конкретных категорий и регионов.



Выбор визуализации

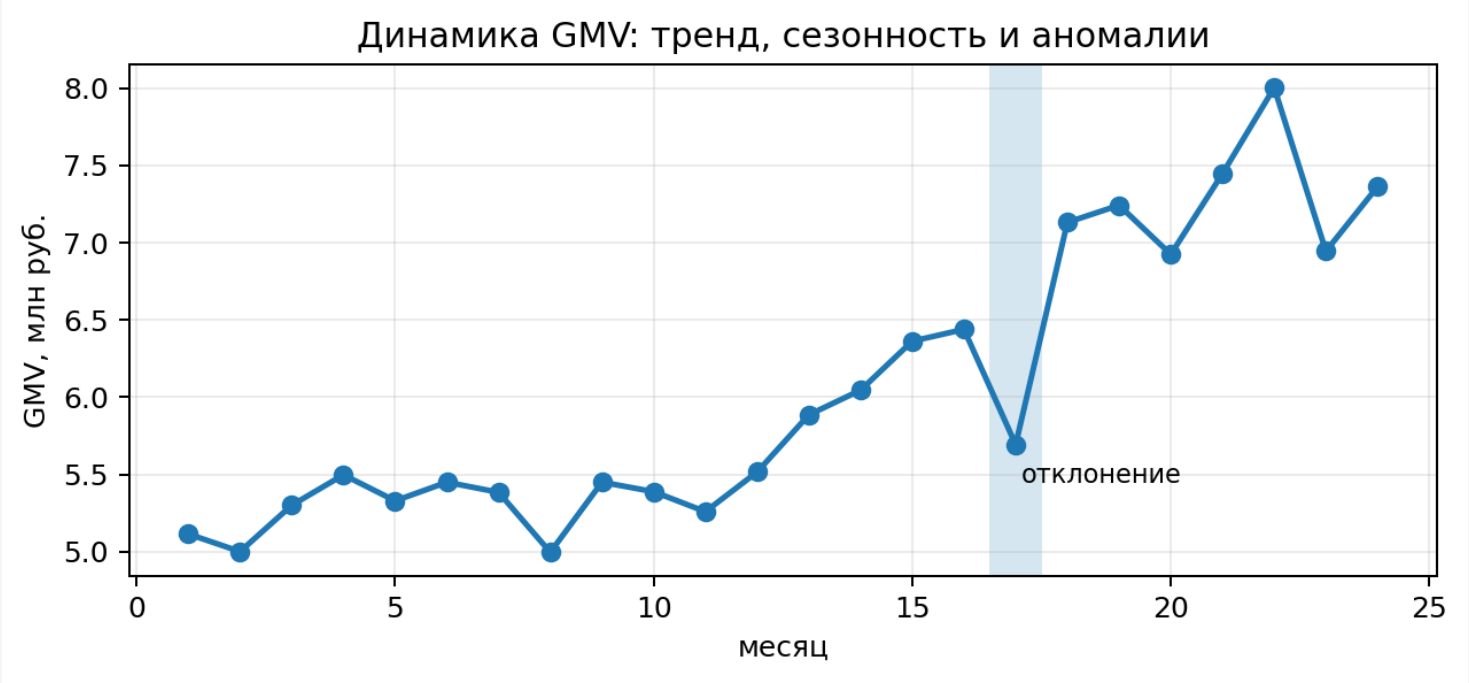
Аналитический вопрос	Тип данных	Обычно подходит
Как меняется показатель?	временной ряд	line chart
Какая категория больше?	категории + мера	bar chart
Как распределены значения?	числовая величина	histogram, boxplot
Есть ли связь между величинами?	две числовые величины	scatter plot
Каков вклад частей?	структура целого	stacked bar, pie с осторожностью

Правило

Тип графика выбирается не по эстетике, а по структуре данных и аналитическому вопросу.

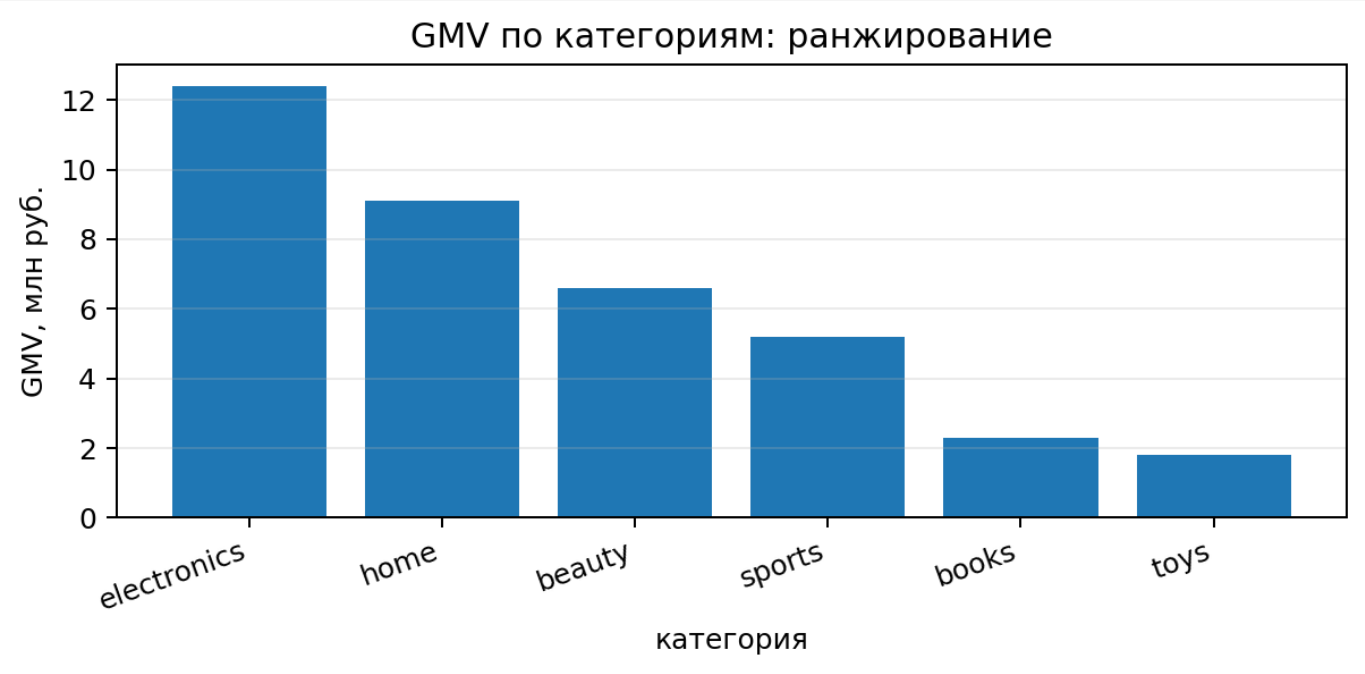


Временной ряд – тренд, сезонность, откло-нения



Line chart (график линий) нужен не для украшения отчёта, а для обнаружения динамики, а именно тренда, сезонности, разрывов и аномалий.

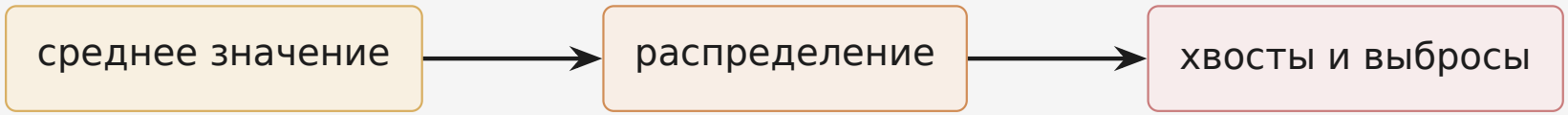
Категориальное сравнение



Bar chart (столбчатый график) хорошо работает, когда нужно сравнить дискретные категории по одной мере.

Почему распределение важнее среднего

Среднее значение удобно, но оно может скрывать неоднородность данных.



Пример

Средняя доставка может выглядеть приемлемой, но часть регионов может иметь крайне длинные задержки. В BI это потребует среза по региону, а в мониторинге — анализа задержек и p95/p99.



Проектирование дашборда

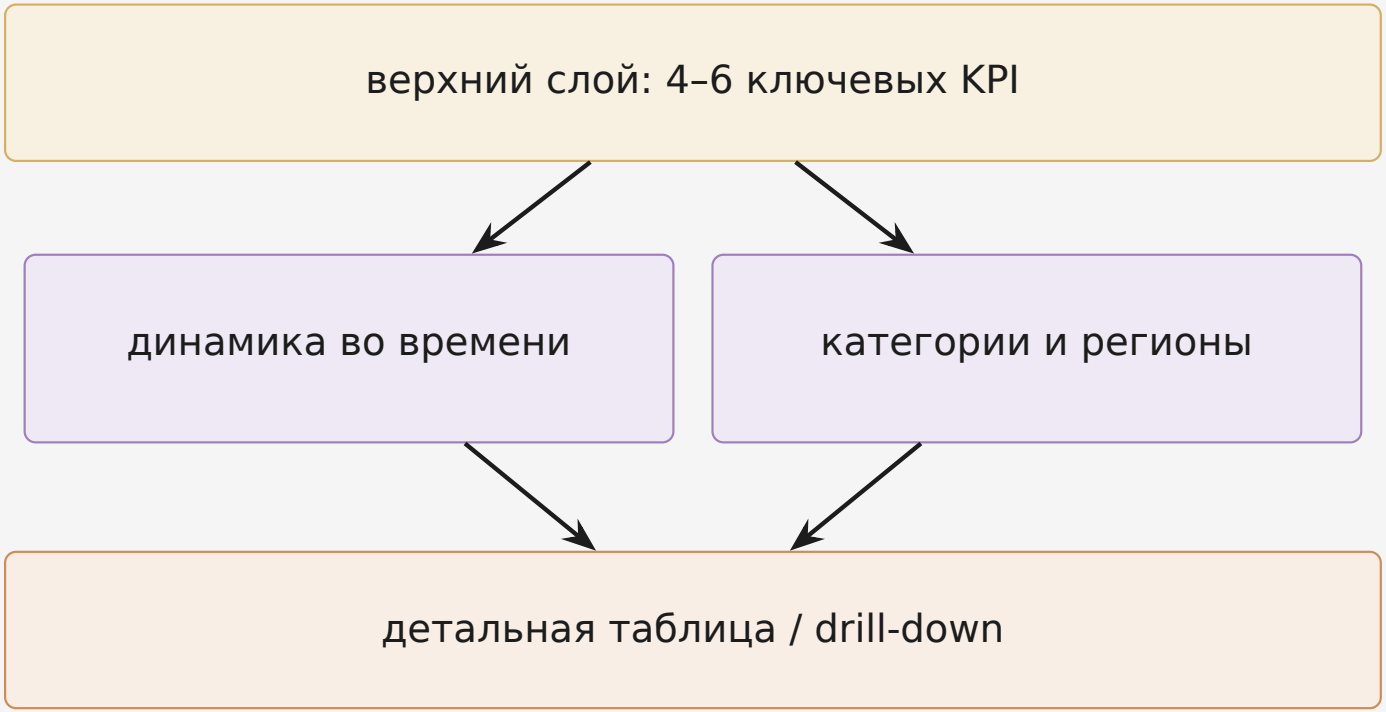
Хороший дашборд является не набором графиков, а управляемой аналитической поверхностью.

Базовые принципы

- сверху — главные KPI и состояние процесса;
- ниже — детализация по ключевым измерениям;
- фильтры должны соответствовать частым аналитическим вопросам;
- цвета и подписи должны кодировать смысл, а не декоративность;
- каждый график должен отвечать на конкретный вопрос.



Иерархия внимания на дашборде



Пользователь сначала должен понять состояние системы, затем перейти к причинам изменения.



Фильтры, drill-down и контекст

Фильтр

Ограничивает данные по периодам, регионам, категориям, сегментам пользователей.

Drill-down

Позволяет перейти от общего показателя к детализации по годам $\rightarrow$ месяцам $\rightarrow$ дням $\rightarrow$ заказам.

Контекст

Показатель без контекста плохо интерпретируем. Нужно показывать базу сравнения по прошлому периоду, плану, среднему, распределениям, целевому порогу.



Опасные визуализации

Типовые ошибки

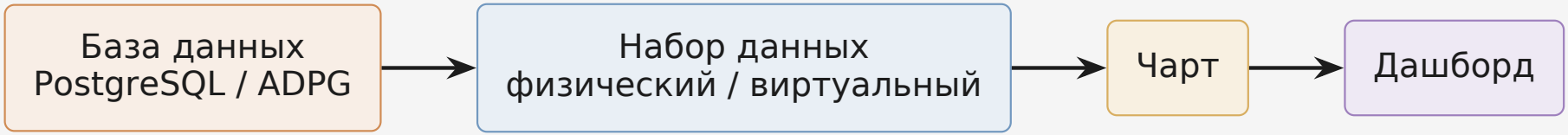
- обрезанная ось без указания;
- слишком много категорий на pie chart;
- смешение абсолютных и относительных значений;
- разные формулы одной метрики;
- перегруженный дашборд без главного вывода.

Научная дисциплина

Визуализация должна повышать проверяемость вывода. Если график усиливает впечатление, но скрывает основание расчёта, он опасен для аналитической системы.



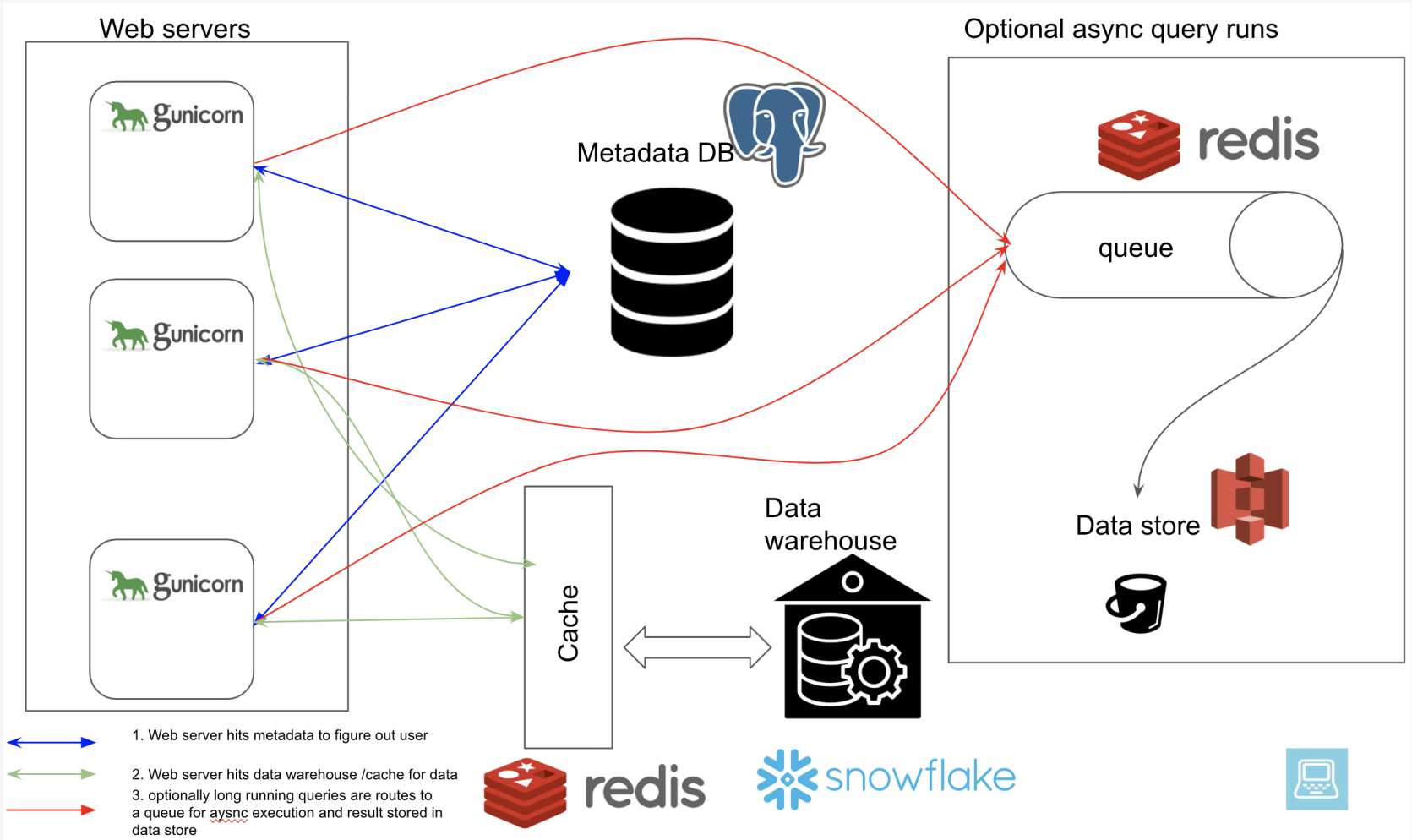
Apache Superset архитектурная модель



Понятия Superset

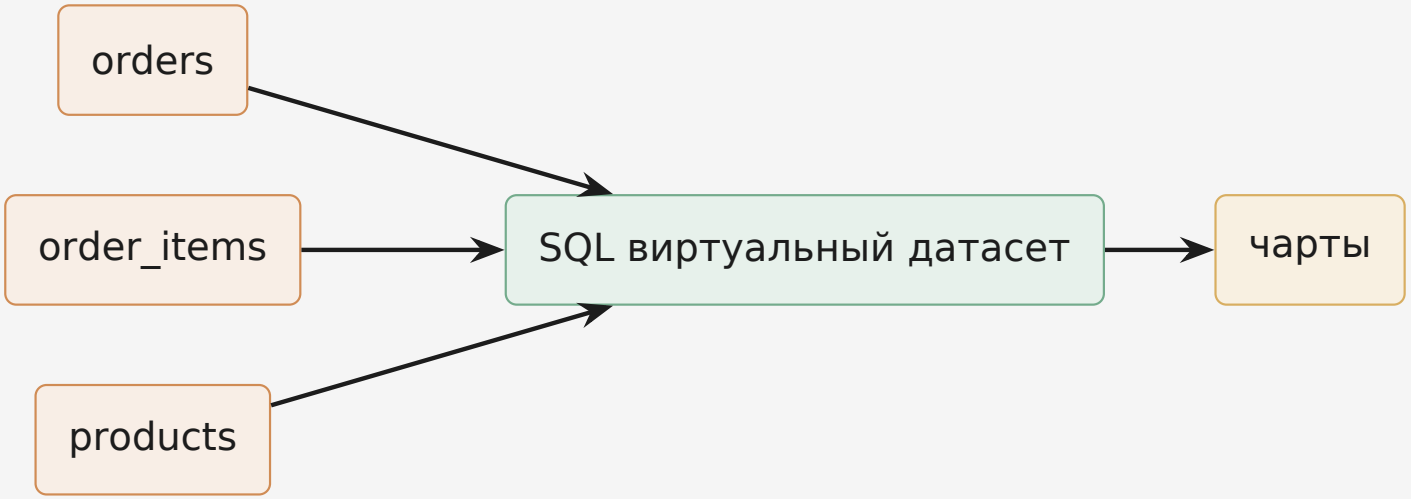
- подключение к БД;
- набор данных: таблица или SQL-запрос;
- чарт как конкретная визуализация;
- дашборд как композиция графиков, фильтров и пояснений.




Web servers
gunicorn
gunicorn
gunicorn
Metadata DB
Optional async query runs
redis
queue
Data store
Cache
Data warehouse
1. Web server hits metadata to figure out user
2. Web server hits data warehouse /cache for data
3. optionally long running queries are routes to a queue for aysnc execution and result stored in data store
redis
snowflake

Виртуальный набор данных в Superset

Virtual dataset позволяет описать источник графиков через SQL-запрос, а не только через физическую таблицу.



Это удобно для учебных задач, но в производственной системе критически важные вычисления лучше фиксировать в управляемых витринах.



Регион / Менеджер

РЕГИОН

Type or Select [Регион]

МЕНЕДЖЕР

Type or Select [Менеджер]

APPLY

Группа / Продукт

ГРУППА

Type or Select [Группа]

ПРОДУКТ

Type or Select [Продукт]

APPLY

Суммарный объем продаж за период, руб

4.96k

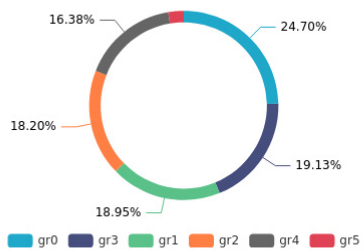
Суммарный объем продаж за период, шт

199

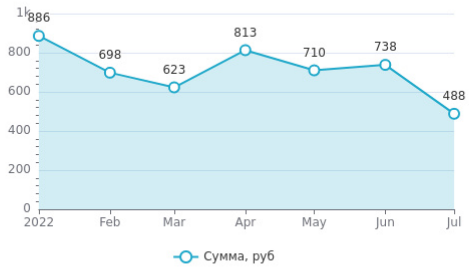
Величина средней продажи за период, руб

24.9

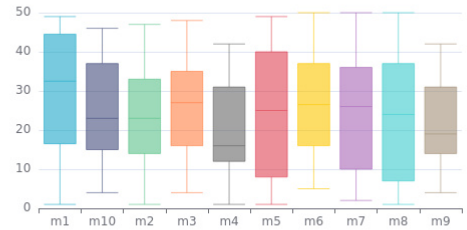
Структура продаж по группам товаров, руб



Динамика по месяцам, руб



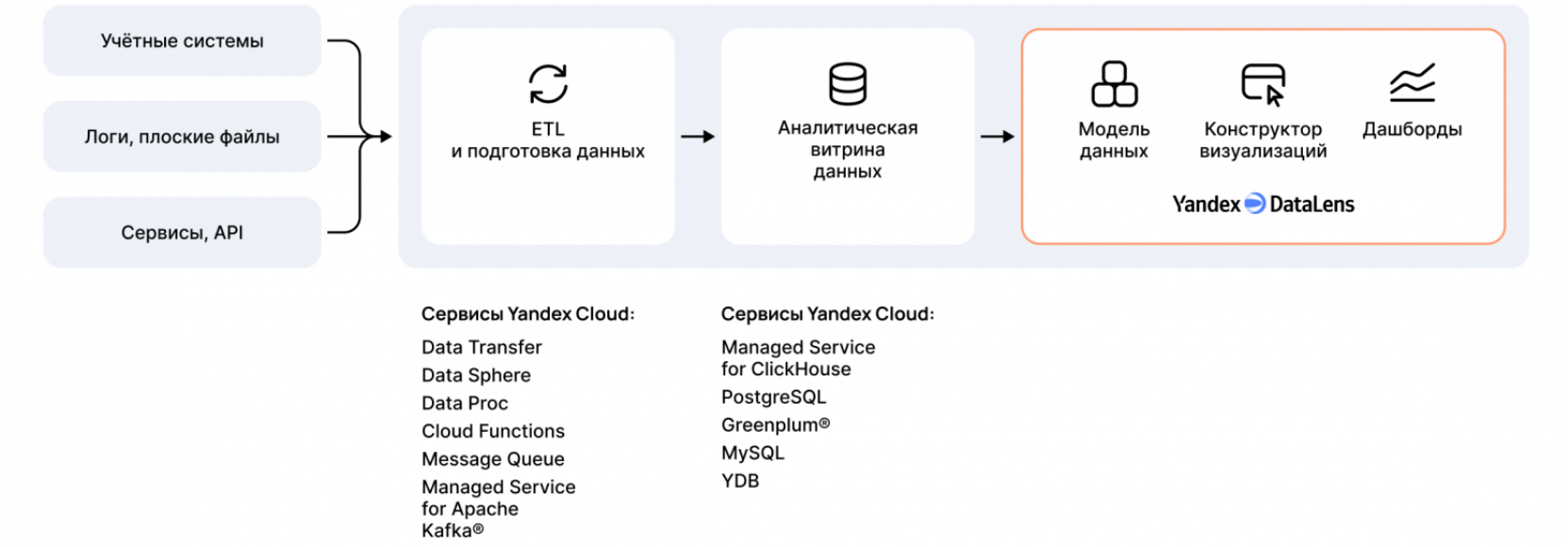
Статистика по менеджерам



Сводная таблица (менеджер-регион)

	metric	Сумма, руб											Кол-во, шт											Среднее, руб											Total (Sum)
	Менеджер	m1	m2	m3	m4	m5	m6	m7	m8	m9	m10	Subtotal	m1	m2	m3	m4	m5	m6	m7	m8	m9	m10	Subtotal	m1	m2	m3	m4	m5	m6	m7	m8	m9	m10	Subtotal	
Регион																																			
r1		82	26	92	9	66	32	98	40	55	21	**521**	4	1	4	1	4	2	2	2	3	2	**25**	20.5	26	23	9	16.5	16	49	20	18.33	10.5	**208.83**	**754.83**
r2		44	18	133	14	71	59	86	37		88	**550**	1	1	6	1	3	3	3	1		3	**22**	44	18	22.17	14	23.67	19.67	28.67	37		29.33	**236.5**	**808.5**
r3		171	38	76	82	31	53	7	67	42	89	**656**	4	2	2	3	3	2	1	3	2	4	**26**	42.75	19	38	27.33	10.33	26.5	7	22.33	21	22.25	**236.5**	**918.5**
r4		49	130	44	130	180	82	68	122	95	28	**928**	1	6	2	6	6	2	3	3	3	1	**33**	49	21.67	22	21.67	30	41	22.67	40.67	31.67	28	**308.33**	**1.27k**
r5		61	63		41	213	148	98	71	18	127	**840**	3	3		1	6	5	5	4	1	4	**32**	20.33	21		41	35.5	29.6	19.6	17.75	18	31.75	**234.53**	**1.11k**
r6			21	68	20	11	108	162	1	65	79	**535**		2	3	2	1	4	5	1	3	3	**24**		10.5	22.67	10	11	27	32.4	1	21.67	26.33	**162.57**	**721.57**
r7		68	140	131	59	145	55	111	66	14	137	**926**	3	4	4	3	5	2	6	3	1	6	**37**	22.67	35	32.75	19.67	29	27.5	18.5	22	14	22.83	**243.92**	**1.21k**
	Total (Sum)	**475**	**436**	**544**	**355**	**717**	**537**	**630**	**404**	**289**	**569**	**4.96k**	**16**	**19**	**21**	**17**	**28**	**20**	**25**	**17**	**13**	**23**	**199**	**199.25**	**151.17**	**160.58**	**142.67**	**156**	**187.27**	**177.83**	**160.75**	**124.67**	**171**	**1.63k**	**6.79k**

Yandex DataLens



Ключевое сходство

Superset и DataLens различаются интерфейсом и экосистемой, но логика BI одинакова, присутствуют подключение, модель данных, визуализация, дашборд.



Концепция важнее конкретного GUI

Интерфейс меняется быстрее, чем принципы построения аналитической модели

Стабильные понятия

- источник данных;
- набор данных или семантический слой;
- измерение и мера;
- фильтр;
- чарт;
- дашборд;
- права доступа и публикация.



BI-антипаттерны

Технические

- тяжёлые JOIN прямо в каждом графике;
- отсутствие единой витрины;
- непроверенные вычисляемые поля;
- ручная правка экспортированных таблиц.

Аналитические

- KPI без цели;
- график без вопроса;
- сравнение несопоставимых периодов;
- среднее без распределения;
- вывод без контекста.



Раздел

Наблюдаемость



Рабочее определение

Наблюдаемость — способность судить о внутреннем состоянии системы по внешне наблюдаемым сигналам.

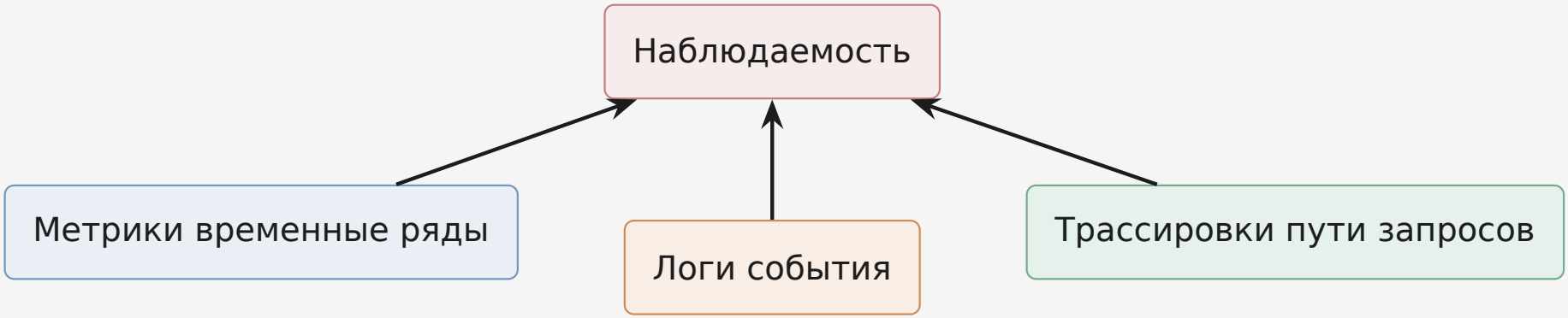
Для аналитической платформы это означает

- видеть состояние БД, Spark-задач, сервисов и инфраструктуры;
- быстро обнаруживать деградацию;
- связывать пользовательский эффект с технической причиной;
- отличать обычную флуктуацию от инцидента.

Мониторинг отвечает на вопрос «что видно сейчас?», наблюдаемость — «можем ли мы объяснить состояние системы по наблюдаемым сигналам?».



Три источника наблюдаемости



В рамках курса

Основной акцент делаем на метриках, потому что Prometheus и Grafana позволяют быстро построить воспроизводимый контур наблюдения.



Метрика как временной ряд

В наблюдаемости метрика почти всегда рассматривается как функция времени.

$$metric : t \mapsto x(t)$$

С учётом меток:

$$metric(t, label_1, label_2, \ldots, label_k).$$

Пример

$$http_requests_total(t, method = \text{“GET”}, status = \text{“500”})$$

Метки не просто украшают метрику. Они задают измерения технического анализа.



Бизнес-метрики и системные метрики

Бизнес-метрики

- gmv;
- orders;
- aov;
- оценки в отзывах;
- конверсия.

Источник — предметные данные.

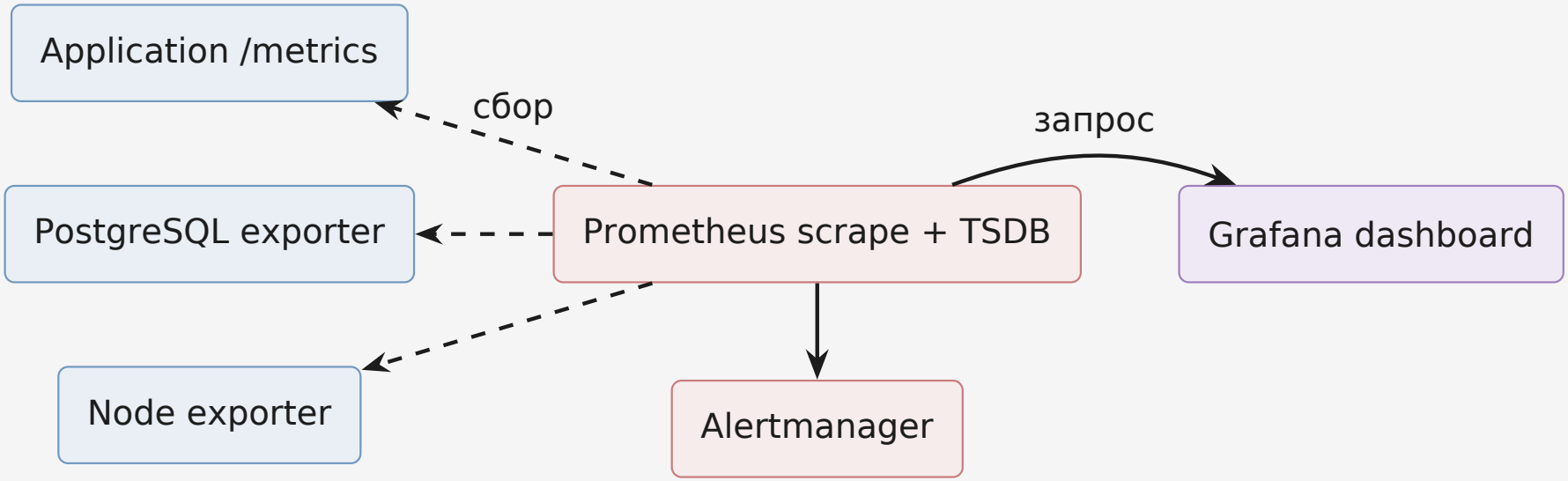
Системные метрики

- cpu_usage;
- memory_usage;
- requests_total;
- query_duration;
- executor_failures.

Источник — работа системы.



Prometheus – базовая архитектура

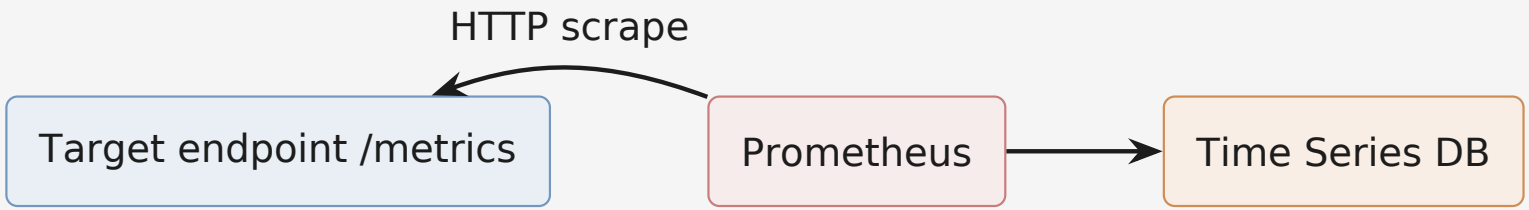


Prometheus обычно сам опрашивает таргеты и сохраняет значения как временные ряды.



Pull-модель Prometheus

В классической конфигурации Prometheus работает по модели **pull**, т.е. он периодически обращается к целевым сервисам и забирает метрики.



Почему это удобно

Центральная система контролирует периодичность опроса, список таргетов и единый формат хранения временных рядов.



Формат метрик Prometheus

```
# HELP http_requests_total Total number of HTTP requests
# TYPE http_requests_total counter
http_requests_total{method="GET",status="200"} 1542
http_requests_total{method="GET",status="500"} 23

# TYPE process_resident_memory_bytes gauge
process_resident_memory_bytes 72839168
```

Состав метрики

- имя метрики;
- набор меток-пар;
- числовое значение;
- timestamp хранится в системе временных рядов.



Имена и меькт

Имя метрики

Должно описывать измеряемую величину: `requests_total`, `query_duration_seconds`, `memory_usage_bytes`.

Метки

Позволяют разделять временные ряды по измерениям: endpoint, method, status, instance, job, database.

`metric{label="value"}` $\rightarrow$ один временной ряд

Слишком большое число уникальных меток-значений порождает проблему высокой кардинальности.



Проблема высокой кардинальности

Если метка принимает слишком много уникальных значений, число временных рядов резко растёт.

$$N_{series} \approx \prod_{j=1}^{k} |L_j|,$$

где $|L_j|$ — число значений j-ой метки.

Опасный пример

`request_duration{user_id="..."}` может создать миллионы временных рядов и перегрузить мониторинг.

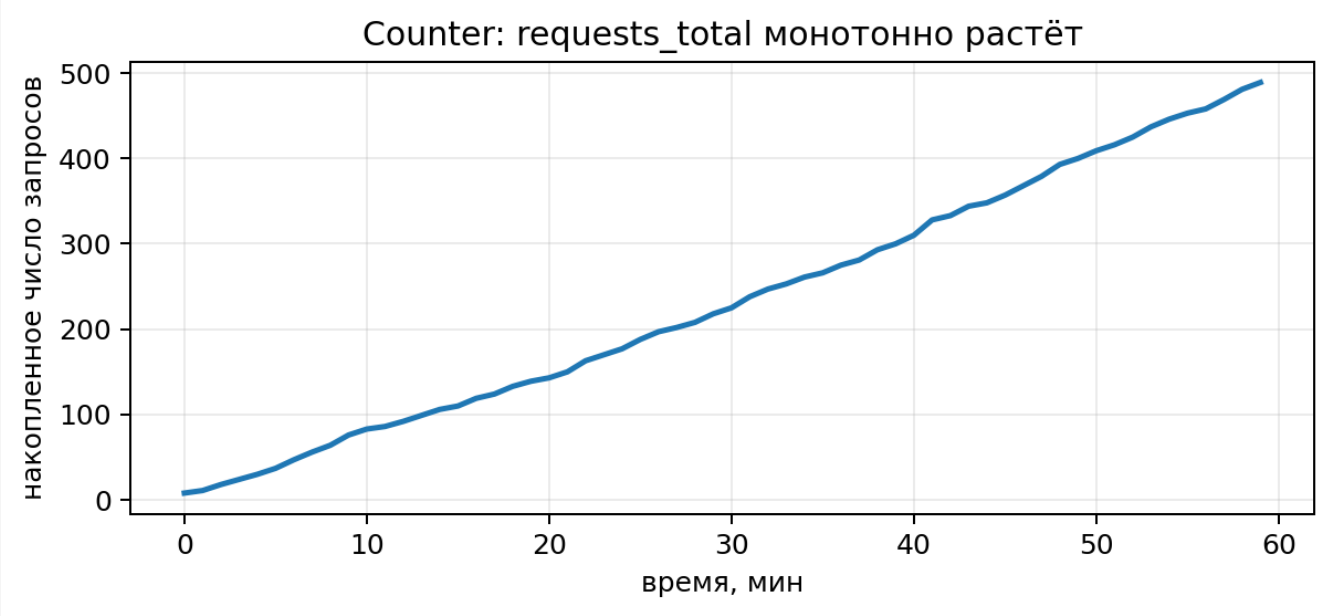
Вывод

Метки должны описывать устойчивые технические измерения, а не произвольные идентификаторы событий.



Счетчики

Счетчик — монотонно возрастающая метрика, которая увеличивается при наступлении события.

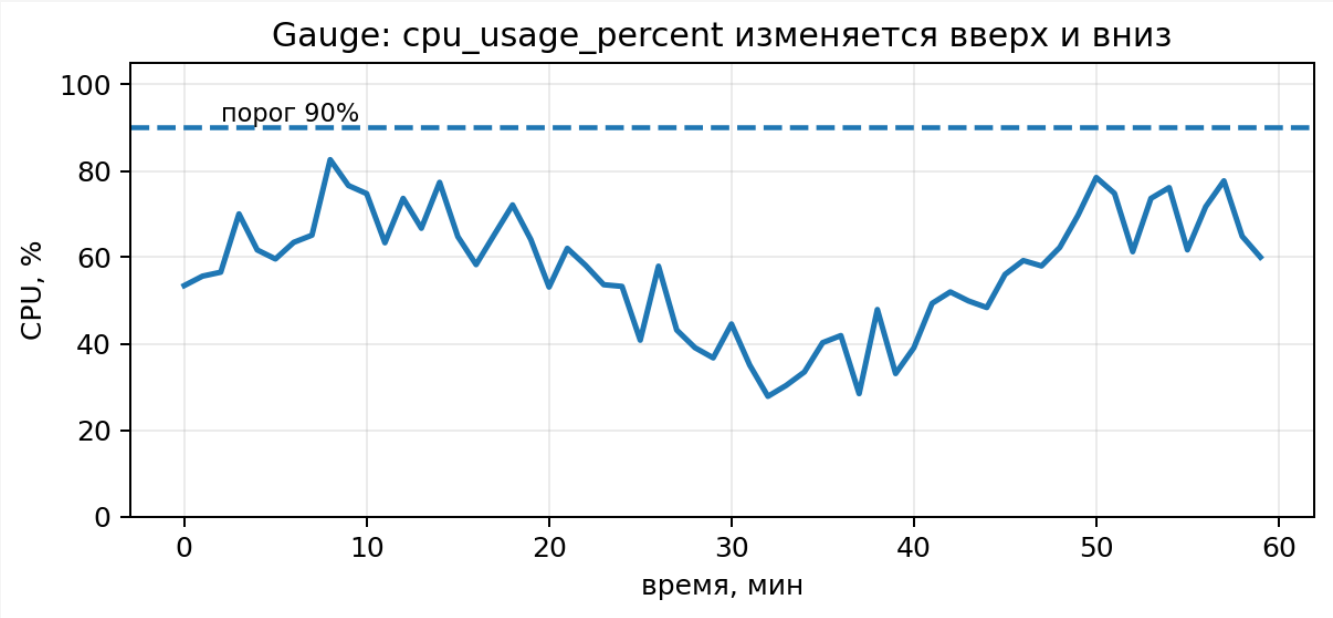


Примеры: `requests_total`, `errors_total`, `processed_rows_total`.



Датчик

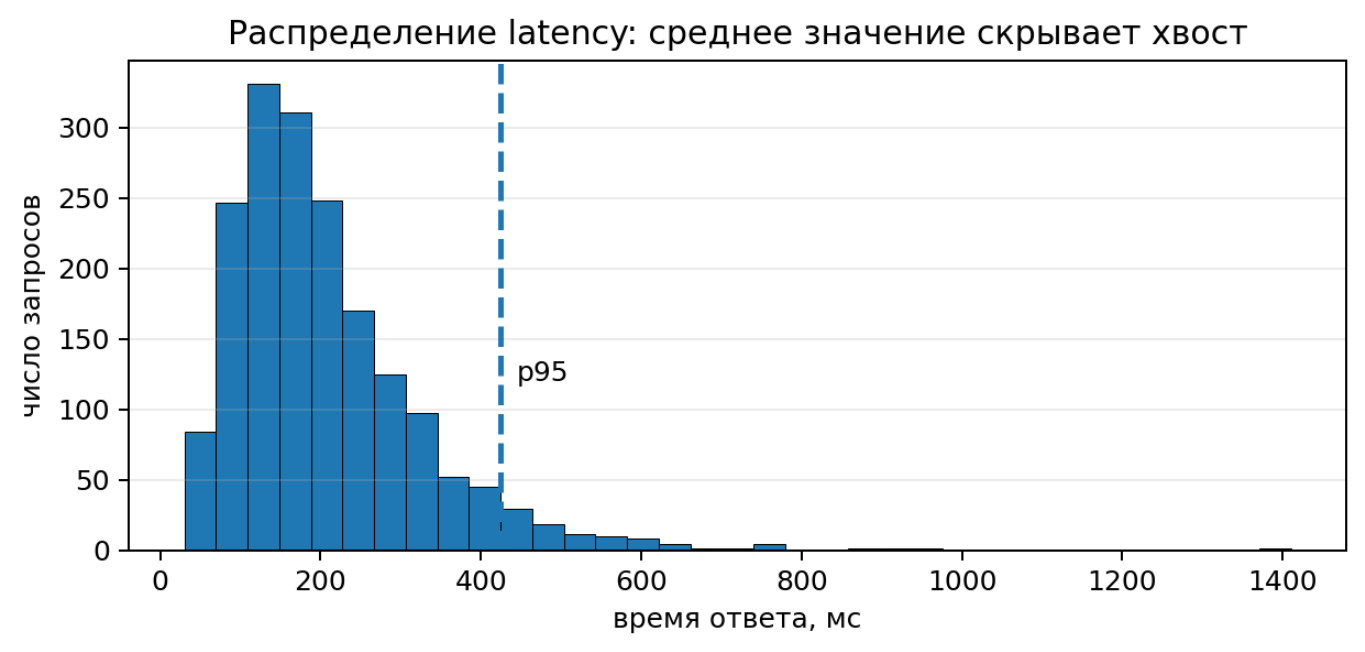
Датчик — метрика, которая может увеличиваться и уменьшаться.



Примеры: CPU usage, memory usage, active connections, queue size.

Гистограмма

Гистограмма накапливает распределение наблюдений по пакетным интервалам.



Для задержки важно видеть не только среднее значение, но и хвост распределения: p95, p99, редкие тяжёлые запросы.



Сводка и квантили

Сводка также используется для распределений, но часто хранит уже вычисленные квантили на стороне приложения.

$$q_p = \inf\{x : F(x) \geq p\}$$

Интерпретация

Если $p95 = 800\,ms$, это означает, что около 95% наблюдений не превышают 800 мс, а оставшиеся 5% находятся правее этого значения.

Практический смысл

Для пользовательского опыта p95 и p99 часто важнее средней задержки.

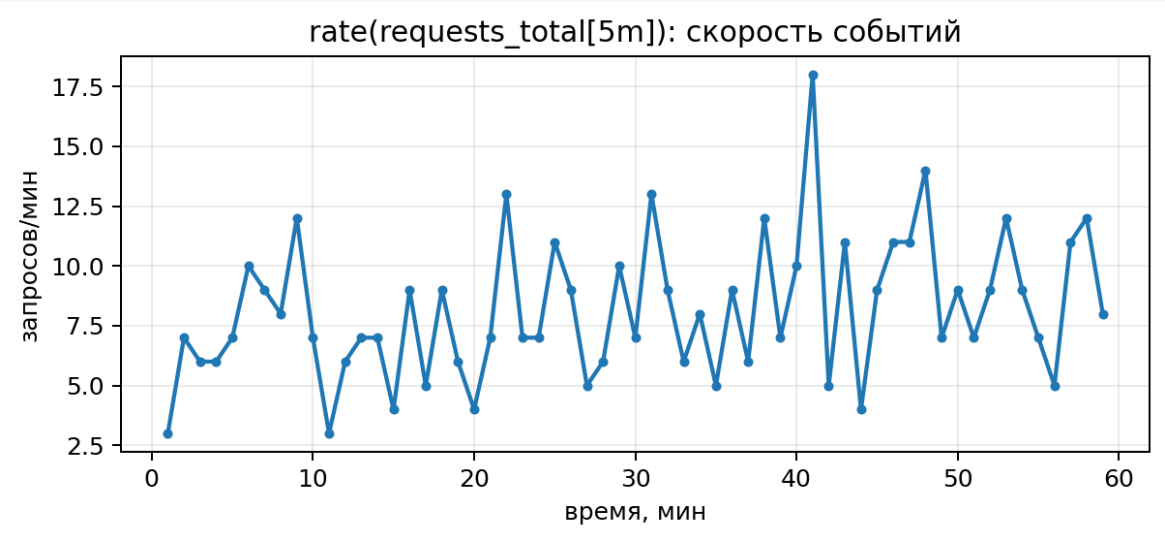


PromQL и зачем нужен rate

Счетчик сам по себе показывает накопленное число событий. Для анализа нагрузки обычно нужна скорость.

$$rate \approx \frac{counter(t_2) - counter(t_1)}{t_2 - t_1}$$

```
rate(http_requests_total[5m])
```



PromQL и агрегация по меткам

```
sum by (status) (
  rate(http_requests_total[5m])
)
```

Смысл запроса

- берём скорость запросов за последние 5 минут;
- группируем временные ряды по меткам `status`;
- суммируем остальные измерения;
- получаем интенсивность ответов 200, 500 и так далее.



PromQL и отношение ошибок

```
sum(rate(http_requests_total{status=~"5.."}[5m]))
/
sum(rate(http_requests_total[5m]))
```

Интерпретация

Это не просто число ошибок, а доля ошибочных ответов в общем потоке запросов. Такая метрика ближе к качеству сервиса.



Правила записи

Правило записи заранее вычисляет часто используемое выражение PromQL и сохраняет результат как новый временной ряд.

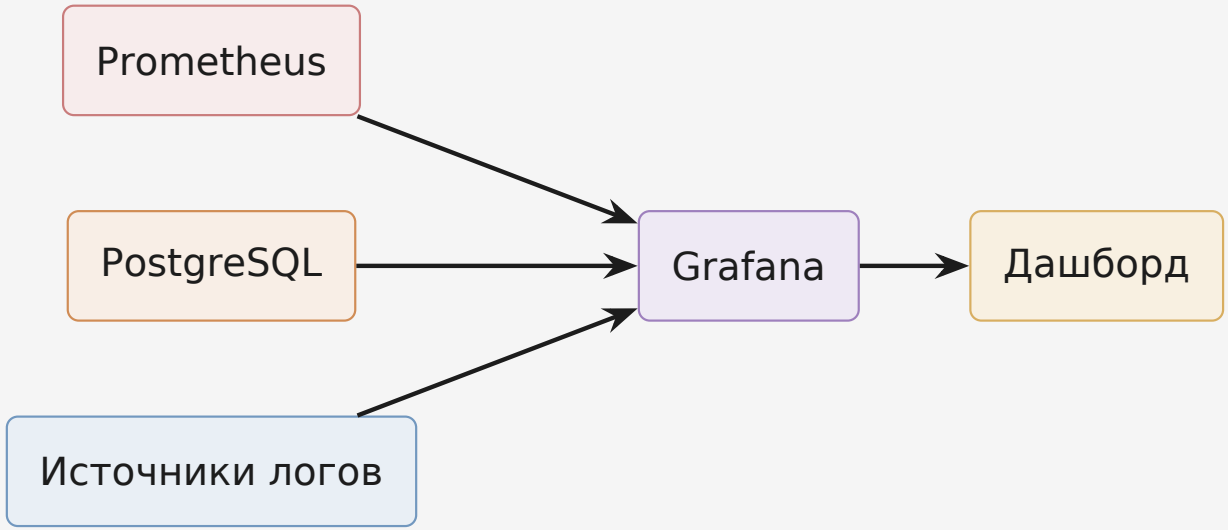
```
record: job:http_requests:rate5m
expr: sum by (job) (rate(http_requests_total[5m]))
```

Зачем это нужно

- ускорить дашборды;
- унифицировать вычисление сложных метрик;
- использовать предрасчитанные ряды в предупреждениях;
- снизить повторение одинаковых запросов.



Grafana: архитектурная роль



Идея

Grafana не обязательно хранит метрики сама. Она подключается к источникам данных и строит панели, запросы, переменные, дашборды и предупреждения.



Технический дашборд аналитической системы

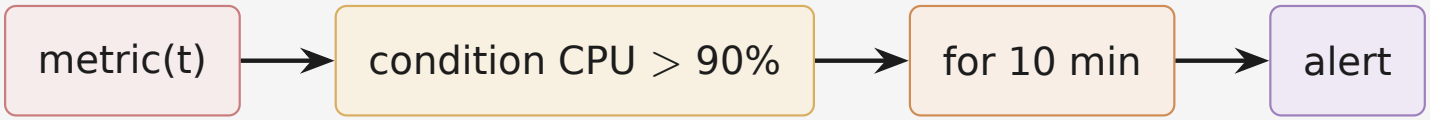
Минимальный набор панелей

- доступность PostgreSQL / ADPG;
- число активных соединений;
- длительность SQL-запросов;
- CPU, RAM, disk IO;
- статус Spark-задач;
- частота ошибок приложения;
- задержка API или BI-сервиса.

Цель технического дашборда — не отчётность, а быстрое обнаружение и объяснение деградации.



Alerting: dashboard сообщает, alert требует действия



Различие

Dashboard показывает состояние. Alert означает, что состояние требует реакции ответственного человека или автоматизированного процесса.



Пример alert-правила

```
- alert: HighErrorRate
  expr: |
    sum(rate(http_requests_total{status=~"5.."}[5m]))
    /
    sum(rate(http_requests_total[5m])) > 0.05
  for: 10m
  labels:
    severity: critical
  annotations:
    summary: "High HTTP 5xx error rate"
```

Важен не только порог, но и длительность нарушения: краткий всплеск и устойчивый инцидент требуют разной реакции.



Alert fatigue

Alert fatigue — ситуация, когда система порождает слишком много малозначимых срабатываний, и люди перестают воспринимать их как сигнал.

Причины

- пороги слишком чувствительны;
- нет интервала `for`;
- alerts дублируют друг друга;
- нет привязки к пользовательскому эффекту;
- уведомления приходят без инструкции действия.

Правило

Хороший alert должен быть actionable: по нему должно быть понятно, кто и что должен сделать.



SLI, SLO и SLA

SLI

Service Level Indicator — измеряемый показатель качества сервиса: latency, availability, error rate.

SLO

Service Level Objective — целевое значение SLI, например 99.5% успешных запросов за 30 дней.

SLA

Service Level Agreement — соглашение с внешними обязательствами и последствиями нарушения.

В курсе достаточно понять связку: измеряем → задаём цель → реагируем на нарушение.



Observability для PostgreSQL и Spark

PostgreSQL / ADPG

- активные соединения;
- длительность запросов;
- блокировки;
- размер БД и таблиц;
- replication lag;
- cache hit ratio.

Spark / PySpark

- длительность job/stage;
- failed tasks;
- shuffle read/write;
- executor memory;
- skew;
- время чтения данных.

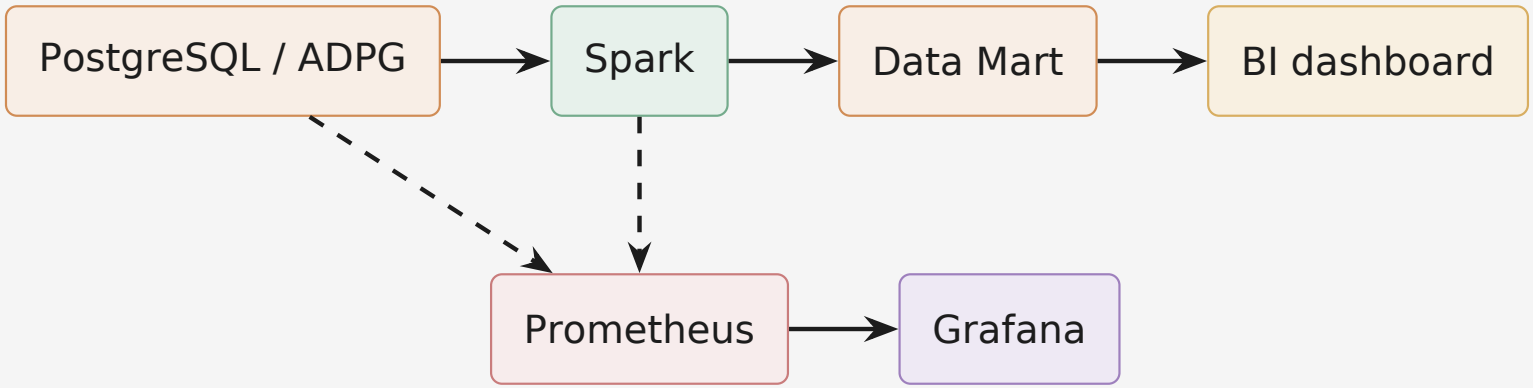


Раздел

Итоговый сценарий визуализации



Два дашборда одной системы

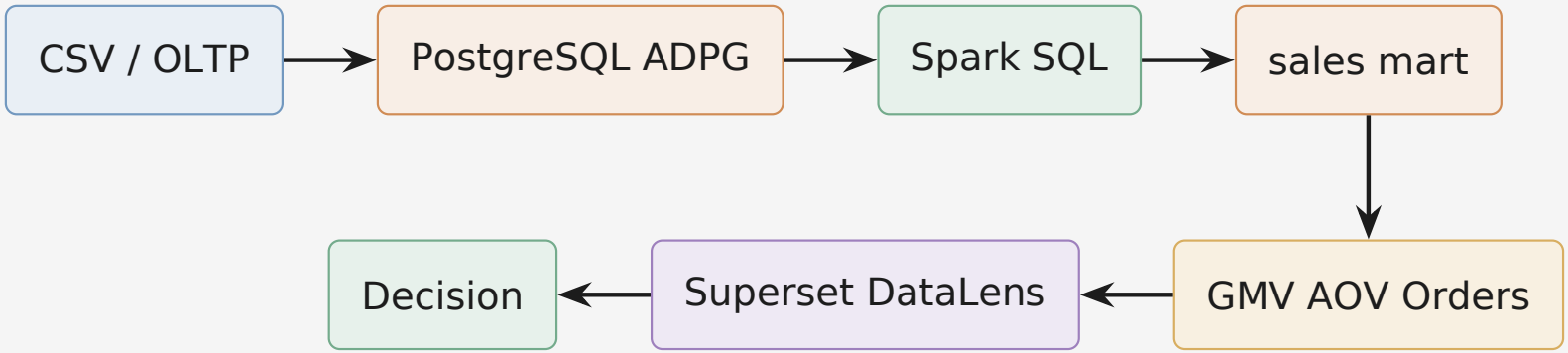


Одна система — две плоскости наблюдения

BI-дашборд показывает бизнес-результат. Grafana-дашборд показывает, можно ли доверять вычислительному контуру и почему он замедлился или отказал.

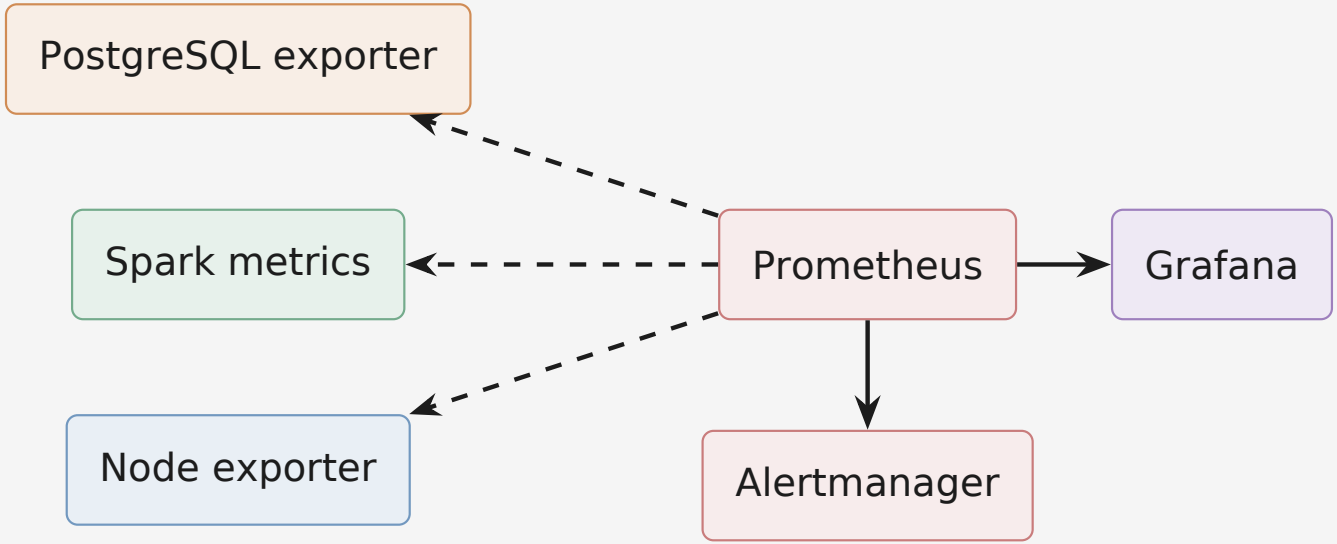


End-to-end контур ecommerce: BI ветка



Здесь данные проходят путь от факта транзакции до управленческого вывода.

End-to-end контур ecommerce: observability ветка

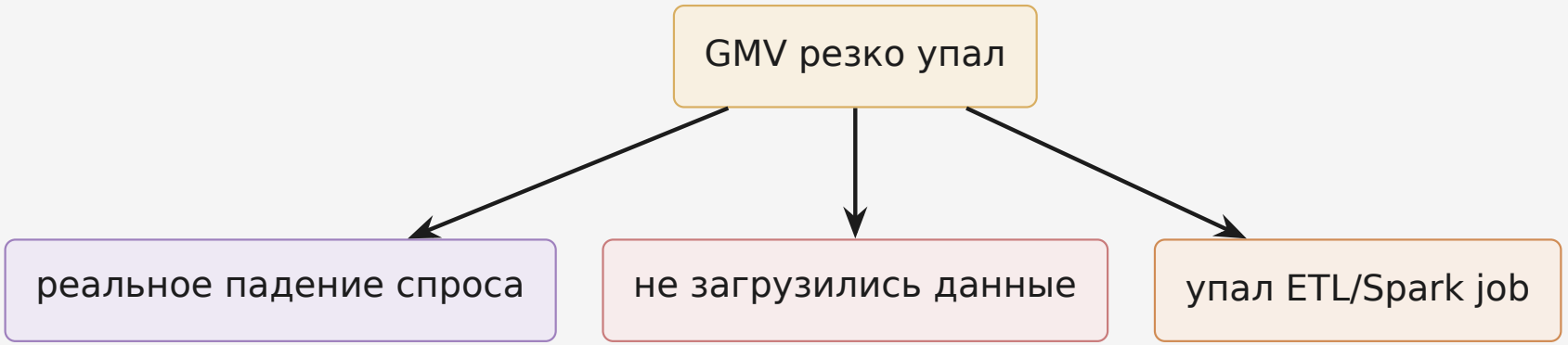


Здесь наблюдаются не продажи, а способность системы эти продажи хранить, обрабатывать и отдавать аналитикам.



Связь бизнес-события и технической причины

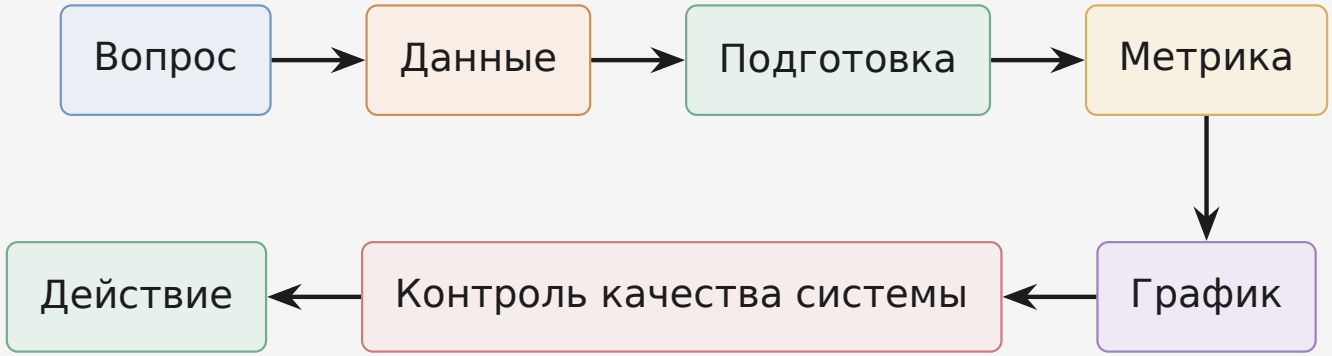
Падение продаж на BI-дашборде может иметь предметную причину, а может быть следствием сбоя обработки данных.



Вывод

BI без observability может принять техническую неполноту данных за бизнес-изменение.

От аналитического вопроса к реализации



Практическая логика

Не начинаем с выбора графика. Начинаем с вопроса, данных, уровня детализации и формального определения метрики.



Контрольные вопросы

1. Почему BI-инструмент не должен заменять витрину данных?
2. Чем metric отличается от KPI?
3. Почему среднее значение может быть опасным без распределения?
4. Чем OLTP-нагрузка отличается от OLAP-нагрузки?
5. Почему label с `user_id` может быть плохой идеей в Prometheus?
6. Когда нужен `rate`, а не сам `counter`?
7. Чем dashboard отличается от alert?
8. Как BI и observability дополняют друг друга в системе `ecommerce`?

